ELEC 475

Lab 2 – Deep Learning Foundations: From LeNet to AlexNet

Connor Leung 20322809

Ryan Su 20344856 | GitHub: https://github.com/connor-leung/elec-475/tree/main/lab2

1. Architectural Specifications

1.1 Base LeNet-5

The Base LeNet-5 is made up of two convolutional (conv) layers followed by a Tanh and a 2x2 AvgPool with three fully connected layers (FC). Base LeNet-5 takes in a 64x64 image as an input: 3×64×64 → conv 5×5 → 6×60×60 → pool → 6×30×30 → conv 5×5 → 16×26×26 → pool → 16×13×13 → flatten (2,704) → 120 → 84 → 10

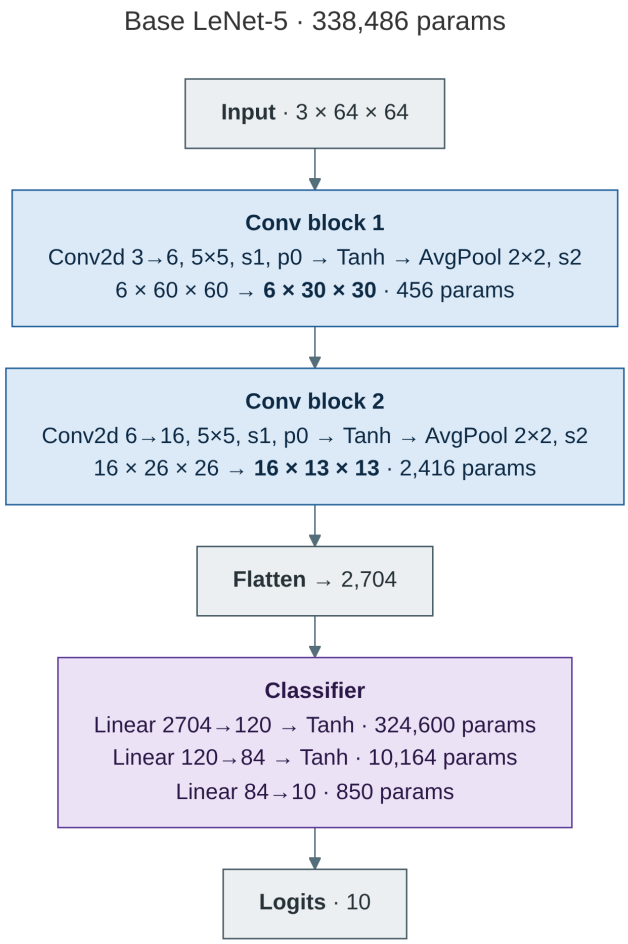


Figure 1: Base LeNet-5 architecture

The size of Base LeNet-5 is 338,486 parameters (params) where the majority (324,600 params) are in FC1. The Base LeNet-5 Convolutional Neural Network (CNN) was created in 1998 and had the issue of using Tanh which saturates and average pools over the input image, causing features to blur.

1.2 Modernized LeNet (ReLU + Overlapping MaxPool)

Modernized LeNet upgrades Base LeNet-5 in two ways: ReLU activation function over Tanh and 3x3 Max Pool with a stride of 2 over 2x2 Average Pool. Because the 3x3

window is larger than the stride, it enables Modernized LeNet windows to overlap, allowing for more information to be retained. The convs are also padded, preserving the spatial size through each conv: 6×64×64 → 6×31×31 → 16×31×31 → 16×15×15. The total params of Modernized LeNet is 446,006 with FC1 having 432,120 params. This is because the flattening becomes 3,600 values instead of 2,704 since the padding keeps the feature map larger.

The conv parameters are unchanged, and the added padding causes FC1 to grow by 107,520 parameters compared to FC1 in Base LeNet-5.

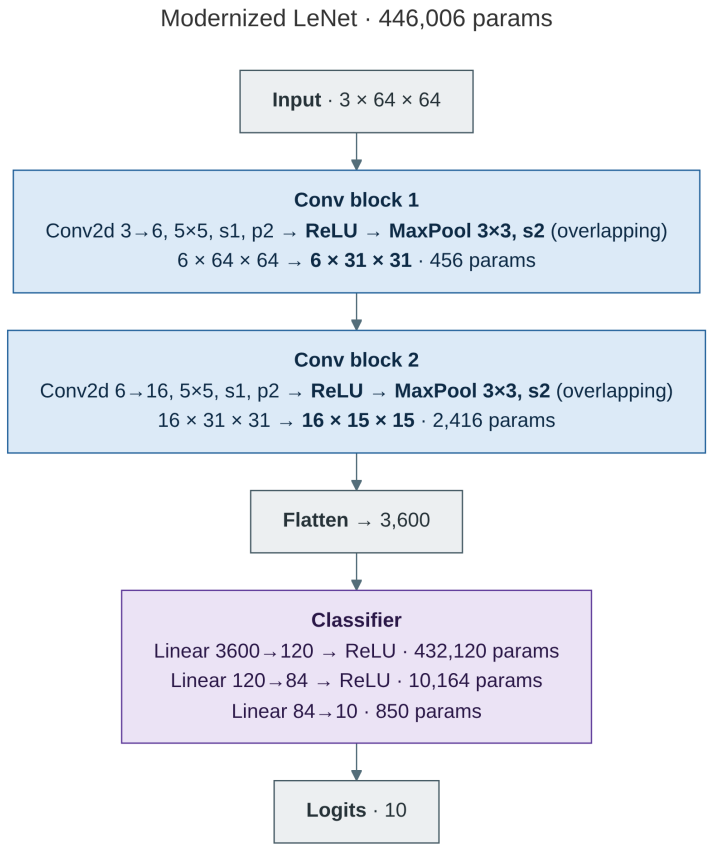


Figure 2: Modernized LeNet architecture

1.3 Full AlexNet-64

AlexNet-64 adapts AlexNet to 64x64 images using five conv layers in five stages. Stage 1 applies a 5x5 conv with a stride of 2 and 64 channels and applies max pooling (64x15x15). Stage 2 has a 5x5 conv with 192 channels with pooling (192x7x7). Stages 3-5 are 3x3 convs with 384, 384, 256 channels and the final pool reduces the output to 256x3x3. The flattened 2,304 features go through a classifier of Dropout(0.5) to 1024 to Dropout(0.5) to 1024 to 10. A normalization layer is put after conv1 and conv2 that uses Identity (none), LocalResponseNorm(5, 1e-4, 0.75, 2), or BatchNorm2d. The BN variant has BatchNorm2d follow conv3-5. Overall the model has 6,608,842 parameters and BN has 6,611,402 (+2,560 BN parameters; LRN adds none). This is higher than the ≈6.3 M quoted in the lab manual, but recomputing each layer from the manual's own

specification (Section 3.2) gives the same 6,608,842, so the difference comes from the manual's reference value rather than our implementation. AlexNet-64 compared to the original AlexNet has the first conv layer shrink from 11x11 with a stride of 4 to 5x5 with a stride of 2, and FC layers shrinking from 4096 to 1024 nodes to have the model take 64x64 inputs.

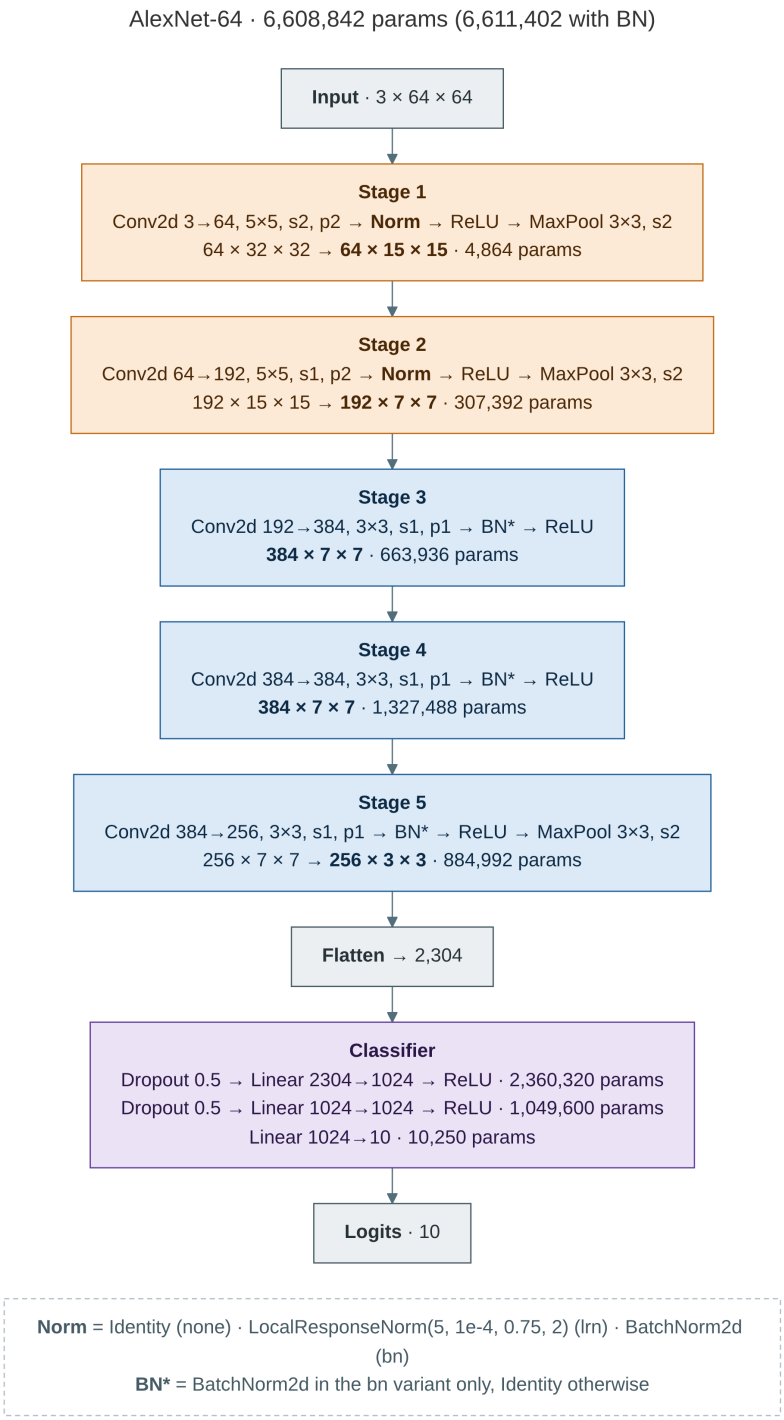


Figure 3: AlexNet-64 architecture (Norm = Identity, LRN or BatchNorm2d)

1.4 PyTorch Model Zoo AlexNet Baseline

A baseline was created using torchvision's AlexNet (torchvision.models.alexnet), trained from scratch without pretrained weights. The last layer is replaced with Linear(4096, 10) for CIFAR-10 or Linear(4096, 200) for Tiny ImageNet. AlexNet was originally designed for 224×224 images, so its first conv layer uses an 11×11 kernel with a stride of 4. On a 64×64 input this shrinks the feature map quickly (64×15×15 after conv1), and by the last pool it is only 256×1×1. AdaptiveAvgPool2d(6, 6) then copies this 1×1 map up to 256×6×6, so the 64×64 input works with the original classifier. The model has 57 million params, with FC1 (9216×4096) alone accounting for 37.8 million. Because the 9,216 features entering FC1 are repeated copies of the same 256 values, most of these parameters receive little useful information.

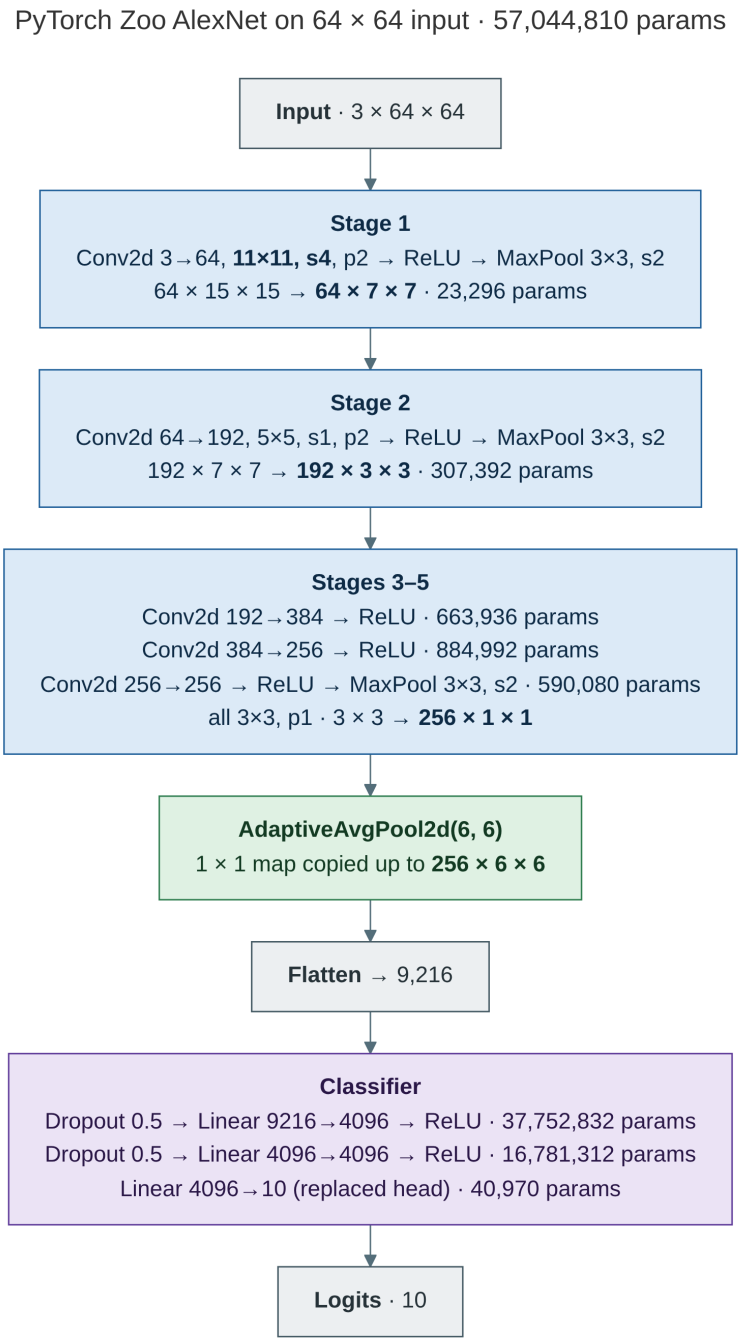


Figure 4: PyTorch Model Zoo AlexNet on a 64x64 input

1.5 Receptive Fields and Parameter Breakdown

AlexNet-64's parameters are split almost evenly between the conv layers and FC layers, a 48:52 ratio, compared to LeNet-5 FC1 having 96% of parameters. The receptive fields (RF) are computed with the formula below where jump is the product of all earlier strides:

$$RF_{out} = RF_{in} + (k - 1) \times jump$$

This gives us the final RF for each of the models we trained:

Table 1

Model	Final RF on a 64×64 input
LeNet-5	16 px
Modern LeNet	19 px
AlexNet-64	97 px, more than the whole image

The AlexNet-64 breakdown:

Table 2

After	RF	Jump
conv1 (5×5, stride 2)	5	2
pool1	9	4
conv2	25	4
pool2	33	8
conv3	49	8
conv4	65	8
conv5	81	8
pool5	97	16

AlexNet-64 parameter split:

Table 3

Part	Params
conv1	4,864
conv2	307,392
conv3	663,936
conv4	1,327,488
conv5	884,992
Conv total	3.19 M (48%)
FC1	2,360,320
FC2	1,049,600

FC3	10,250
FC total	3.42 M (52%)

Each LeNet output only sees a quarter of the image's width, so it can't model entire objects; whereas, AlexNet-64 output, with its improvements, can see the entire image.

2. Empirical Results & Figures

2.1 Exploratory Data Analysis (class balance and feature distributions)

CIFAR-10 has 5,000 training images per class across 10 classes for a total of 50,000 total training images and 10,000 validation images. Tiny ImageNet has 500 per class across 200 classes totalling 100,000 training images and 10,000 validation images. The imbalance ratio is 1.0 with accuracy and macro-F1 following each other closely in all results.

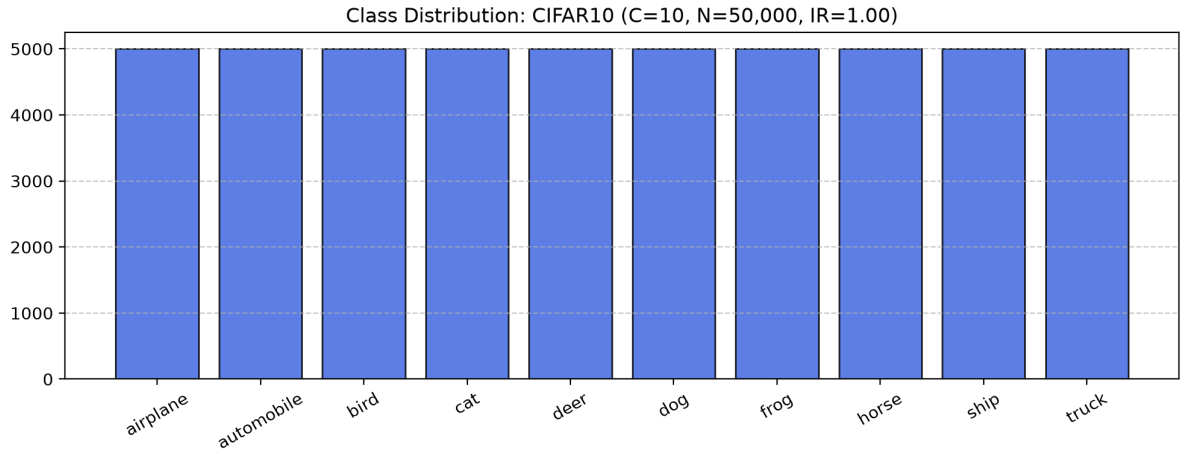


Figure 5: CIFAR-10 class distribution

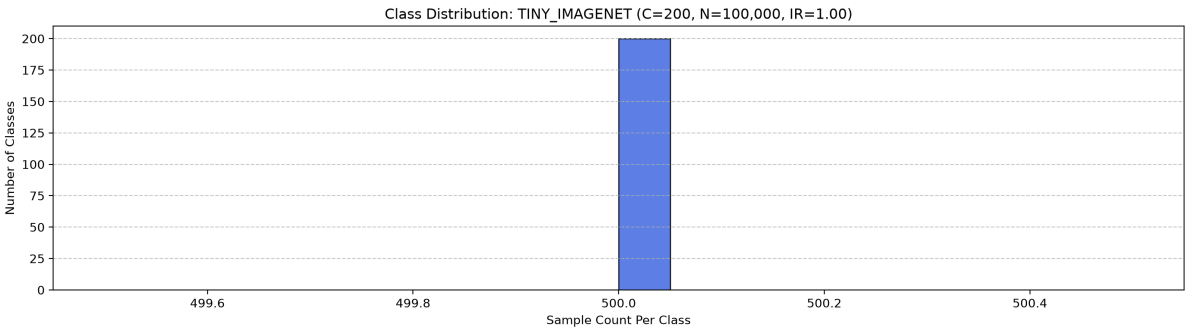


Figure 6: Tiny ImageNet class distribution

Below are the channel histograms and they show that both datasets have similar RGB statistics. Tiny ImageNet is slightly darker in the blue channel with most of its distribution being on the lower end of pixel intensity. Each dataset is standardized with its own per-channel mean and standard deviation: CIFAR-10 uses (0.491, 0.482, 0.447) / (0.247, 0.244, 0.262), and Tiny ImageNet uses (0.480, 0.448, 0.398) / (0.276, 0.269, 0.282).

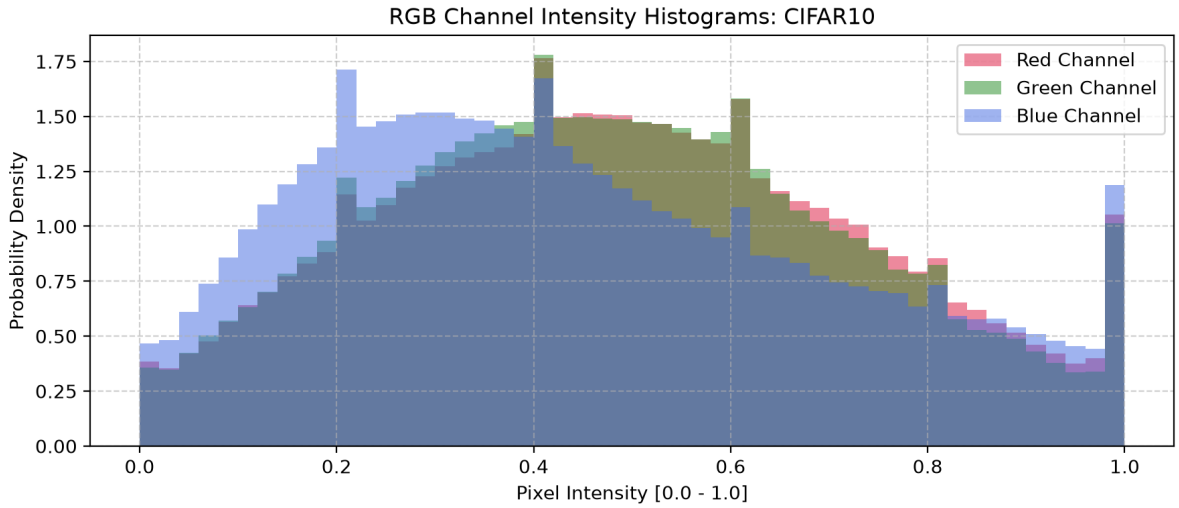


Figure 7: Per-channel pixel intensity histograms (CIFAR-10)

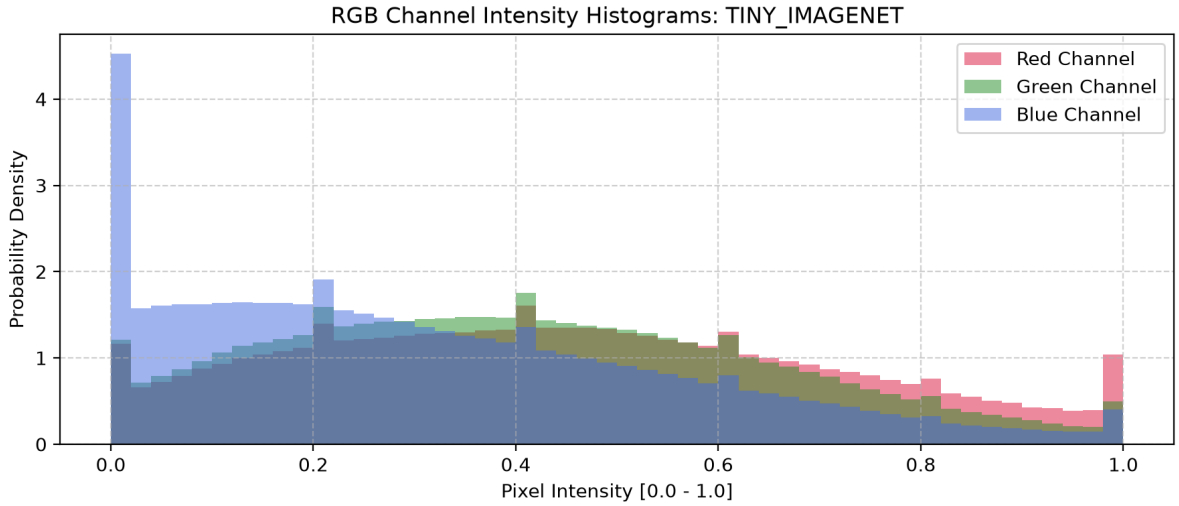


Figure 8: Per-channel pixel intensity histograms (Tiny ImageNet)

2.2 Data Pipeline Verification (demo_transforms.py output)

Testing against OpenCV and NumPy, they produce channels-last float32 arrays of shape (64, 64, 3) with values in [0, 1], stored in host RAM. The PyTorch pipeline produces a channel-first torch.float32 tensor of shape [3, 64, 64] on cuda:0, standardized to roughly [-2, 2]. Since the layouts and value ranges are different,

ToTensor and Normalize are required to fit the tensors into the correct orientation and scale. PyTorch convolutions expect a CxHxW layout and zero-centred inputs. In the 1,000-image benchmark, the GPU pipeline (161.7 ms) was slower than the OpenCV (8.4 ms) and NumPy (26.2 ms). The noticeable time difference is most likely caused by the kernel-launch overhead dominating smaller images when they are processed one at a time and the GPU timing including first-call warm-up. The CPU timings also exclude the time it takes the host-to-device copy that training would still require.

```
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :    8.42 ms  (118,791 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   26.19 ms  ( 38,175 img/s) + Host-to-Device Copy required
- PyTorch (GPU Tensors)   :  161.67 ms  (  6,186 img/s) [ZERO PCIe Transfer Overhead]
================================================================================
```

Figure 9: Output of demo_transforms.py

2.3 Model Complexity Benchmark (compare_models.py table)

In the figures below, Tiny ImageNet has higher parameter counts than CIFAR-10 because the final layer has 200 outputs instead of 10. The Latency is the median single-image inference time from the GTX 1070. FLOPs are counted as 2 × MACs, the same convention as the lab manual. Our values are higher than the manual's reference values (e.g., 4.8 MFLOPs for LeNet-5), but counting each layer by hand from the manual's own specification gives 3.58 M MACs for LeNet-5 and 218.3 M for AlexNet-64, matching our profiler, so the gap comes from the reference values. Under this

convention the Zoo AlexNet needs fewer FLOPs than AlexNet-64 at 64×64, because its stride-4 first layer shrinks the feature maps much earlier.

```
======================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (NVIDIA GeForce GTX 1070)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
======================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.26         | 3919.0             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.26         | 3884.8             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 0.71         | 1410.0             |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 0.91         | 1104.3             |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 0.76         | 1309.6             |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 1.63         | 612.6              |

======================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
======================================================================
```

Figure 10: compare_models.py output for CIFAR-10

```
======================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (NVIDIA GeForce GTX 1070)
Dataset       : TINY_IMAGENET (200 classes)
Input Shape   : (1, 3, 64, 64)
======================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|------------------- |
| 1. LeNet-5 (Base)           | 0.355      | 3.60     | 7.20      | 0.27         | 3718.6             |
| 2. LeNet-Modern (ReLU+Pool) | 0.462      | 4.61     | 9.22      | 0.26         | 3799.8             |
| 3. AlexNet-64 (None)        | 6.804      | 218.54   | 437.08    | 0.67         | 1498.0             |
| 4. AlexNet-64 (LRN)         | 6.804      | 218.54   | 437.08    | 0.88         | 1133.9             |
| 5. AlexNet-64 (BatchNorm)   | 6.806      | 219.18   | 438.36    | 0.72         | 1387.4             |
| 6. Torchvision Zoo AlexNet  | 57.823     | 94.88    | 189.76    | 1.61         | 619.8              |

======================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
======================================================================
```

Figure 11: compare_models.py output for Tiny ImageNet

2.4 Master Ablation Summary (Tables 7.3a and 7.3b)

Table 4: Architecture and cost for CIFAR-10

Model	Conv Depth	Normalization & Regularization	Data Augmentation	Params (M)
1. LeNet-5	2	None	None	0.338
2. Modern LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446
3. AlexNet None	5	Dropout (p = 0.5)	None	6.609
4. AlexNet LRN	5	LRN + Dropout (p = 0.5)	None	6.609
5. AlexNet BN	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611
6. AlexNet BN+Aug	5	BatchNorm2d + Dropout (p = 0.5)	Crop + Flip + ColorJitter	6.611*
7. Zoo AlexNet	5	Dropout (p = 0.5), AdaptiveAvgPool2d	None	57.045

Table 4 (continued): Cost for CIFAR-10

Model	MFLOPs	Mean Epoch Time (s)	Latency (ms)
1. LeNet-5	7.17	9.68	0.26
2. Modern LeNet	9.19	9.88	0.26
3. AlexNet None	436.70	13.42	0.71
4. AlexNet LRN	436.70	21.70	0.91
5. AlexNet BN	437.97	14.94	0.76
6. AlexNet BN+Aug	437.97*	21.38	0.76*
7. Zoo AlexNet	188.20	20.45	1.63

*BN + Aug uses the same network as BN, so params, FLOPs, and latency are identical. Mean epoch time is averaged over 10 seed-42 epochs on a GTX 1070.

Table 5: Validation accuracy and macro-F1

Model	CIFAR-10 Acc	CIFAR-10 F1	Tiny Acc	Tiny F1
1. LeNet-5	55.97	55.81	12.40	10.61
2. Modern LeNet	67.40	67.51	20.28	18.70
3. AlexNet None	72.33	72.30	14.39	12.03
4. AlexNet LRN	74.10	74.17	14.55	12.00
5. AlexNet BN	78.34	77.99	**30.14**	**28.89**
6. AlexNet BN+Aug	**81.07**	**81.05**	28.33	27.08
7. Zoo AlexNet	63.99	63.75	13.32	10.66

2.5 Extended Ablation Experiments (with Δ comparisons)

Five additional experiments were conducted against the AlexNet BN baseline (LR= 1e-3, batch 128, 10 epochs, no augmentation, seed 42). In Table 6, their deltas are measured against the baseline's best validation accuracy of 78.34% and best macro-F1 of 77.99%. The configurations tested are as follows: AlexNet BN, LR = 1e-2, LR = 1e-4, Batch = 64, Batch = 256, and BN + Aug, 30 epochs. The first four change one setting each; the last changes two (augmentation and training length) to show their combined effect.

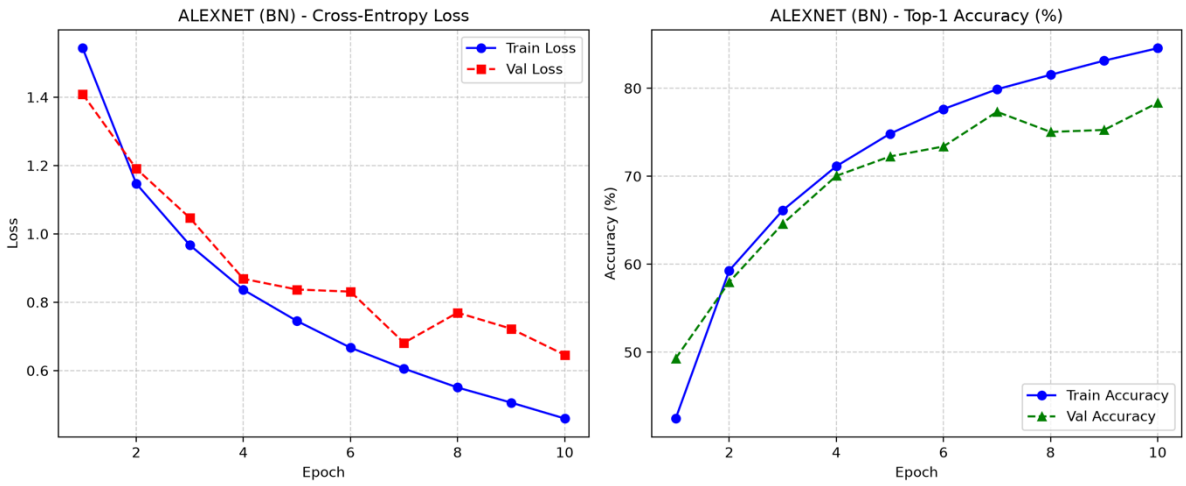


Figure 12: AlexNet BN

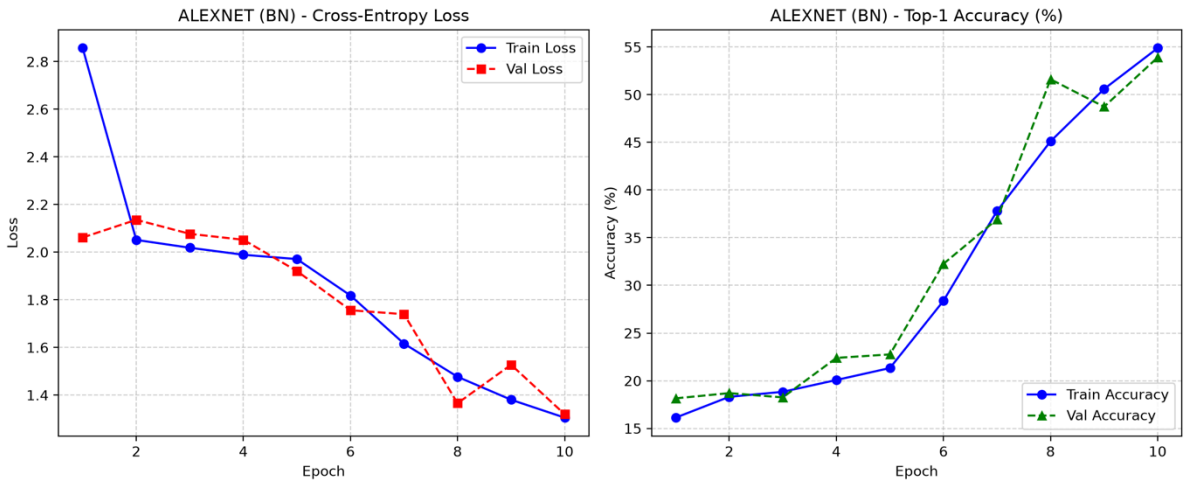


Figure 13: LR 1e-2

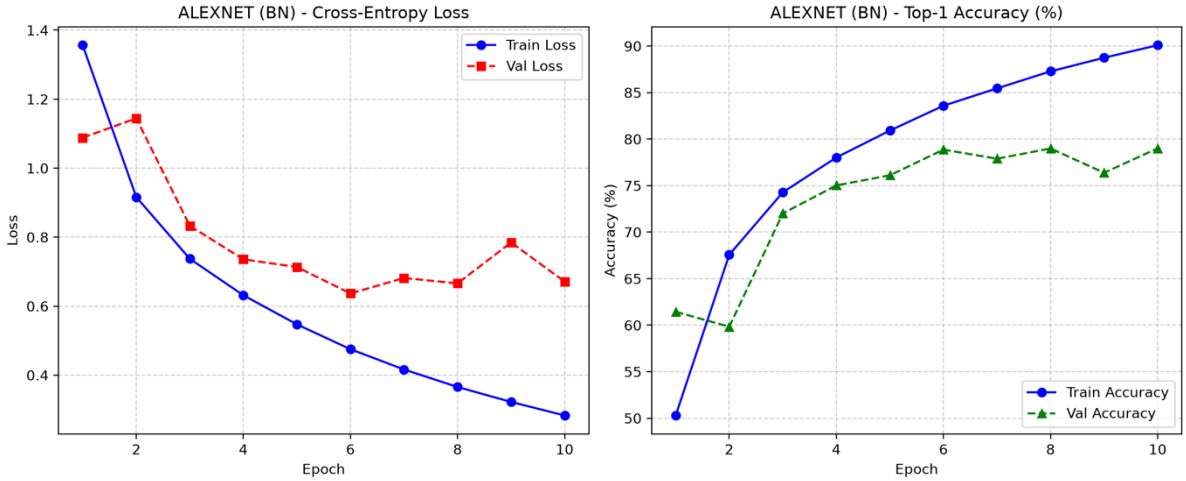


Figure 14: LR 1e-4

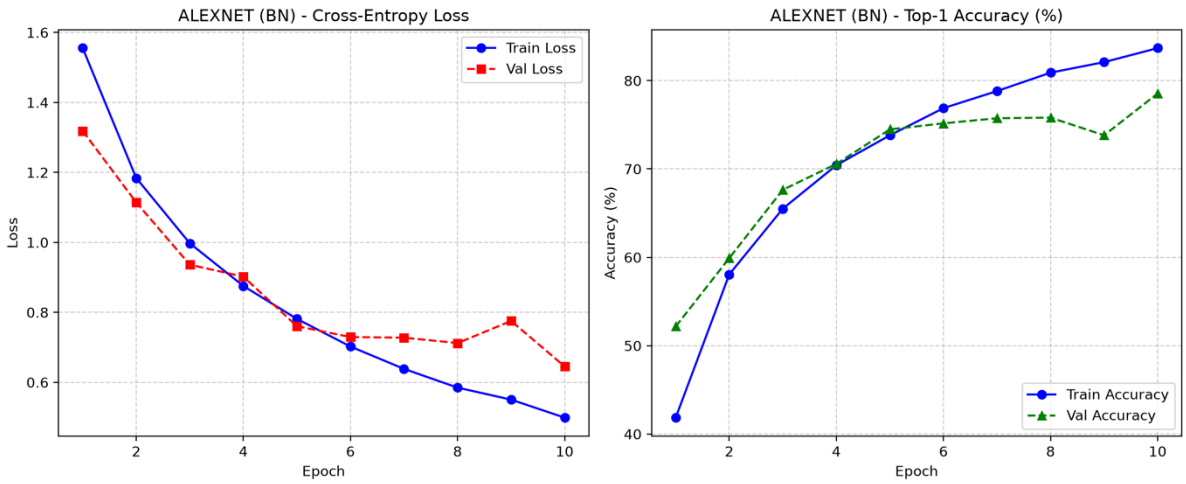


Figure 15: Batch 64

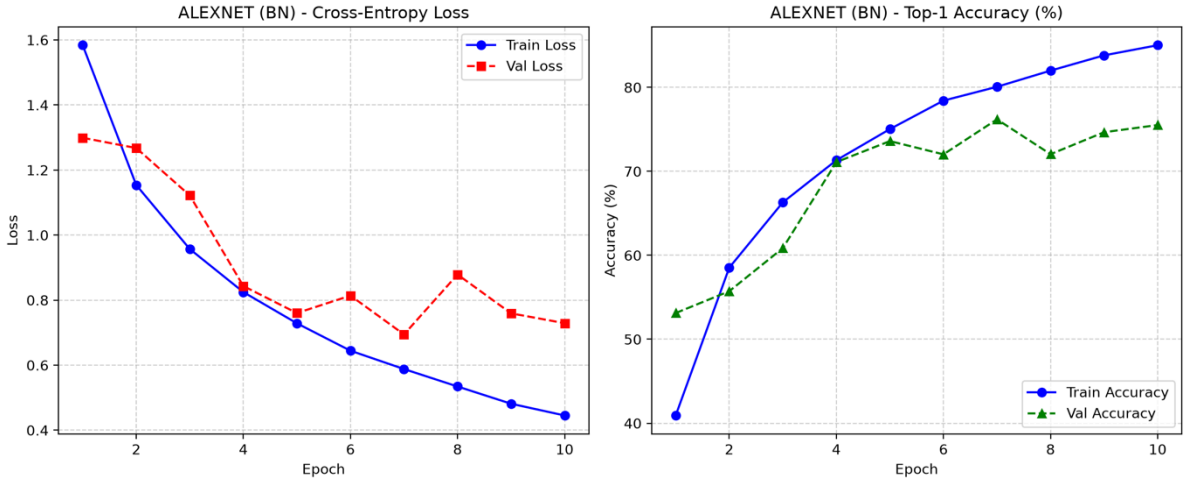


Figure 16: Batch 256

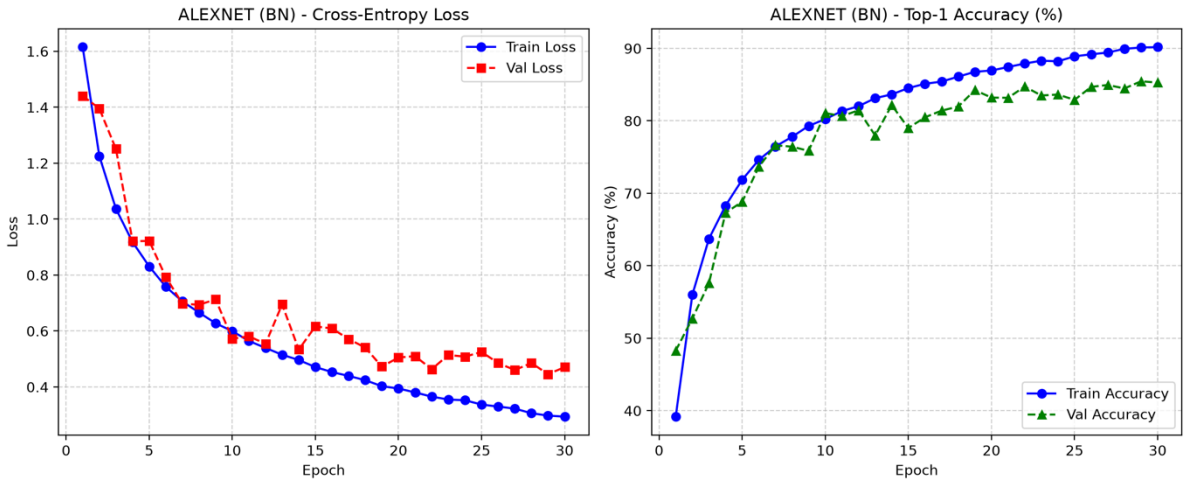


Figure 17: BN + Aug, 30 epochs (early stopping enabled, not triggered)

Table 6: Deltas of results

Experiment	Change	Best val acc (%)	Δ acc (pts)	Best val macro-F1 (%)	Δ F1 (pts)	Mean epoch time (s)
Baseline	n/a	78.34	n/a	77.99	n/a	14.94
LR 1e-2	LR × 10	53.91	−24.43	52.27	−25.72	14.97
LR 1e-4	LR ÷ 10	79.01	+0.67	79.12	+1.13	15.04
Batch 64	Batch ÷ 2	78.54	+0.20	78.54	+0.55	17.09
Batch 256	Batch × 2	76.19	−2.15	76.19	−1.80	14.31
BN + Aug, 30 epochs	Augmentation + 30 epochs (early stopping enabled, not triggered)	85.46	+7.12	85.49	+7.50	21.15

All six runs use the same AlexNet-64 BN network (6.611 M parameters, 437.97 MFLOPs, 0.76 ms latency), so only accuracy, macro-F1 and wall-clock time change. Each run uses seed 42; multi-seed Mean ± Std results for the normalization variants are in Table 7. Batch 64 has the longest epoch time of the 10-epoch runs because it takes twice as many optimizer steps per epoch, and the 30-epoch run is slower per epoch because of CPU-side augmentation.

The high learning rate destabilizes the first epochs and makes the model underfit. It continues to improve across the epochs and is still improving at epoch 10. The low learning rate compared to the baseline has similar accuracy, but its training accuracy climbs to 90% which means the model is overfitting. Changing the batch size to 64 is the same as baseline. Batch 256 halves the number of updates per epoch but has a validation accuracy 2.15 points less than baseline. The significant increase comes from training longer with augmentation, where validation loss keeps falling through to epoch 30 so early stopping is not triggered.

2.6 Multi-Seed Evaluation (Mean ± Std %)

The largest standard deviation (0.84 points) is smaller than the 2-7 point gaps between methods. The order from worst to best normalization method is as follows: None < LRN < BN, and it holds across all three seeds.

Table 7: AlexNet-64 normalization variants on CIFAR-10, mean ± sample standard deviation over seeds 42, 0, and 1 (n = 3).

Norm	Val acc	Macro F1
None	72.08 ± 0.26	71.89 ± 0.40
LRN	74.07 ± 0.31	74.00 ± 0.43
BN	78.74 ± 0.49	78.66 ± 0.84

2.7 Training and Validation Curves

Across the CIFAR-10 runs, BN variants converge the fastest and LeNet-5 converges the slowest. The unaugmented models show validation loss flattening or rising while training loss keeps falling, which is reflected as overfitting. For Tiny ImageNet, only the two BN models separate from the rest in 10 epochs.

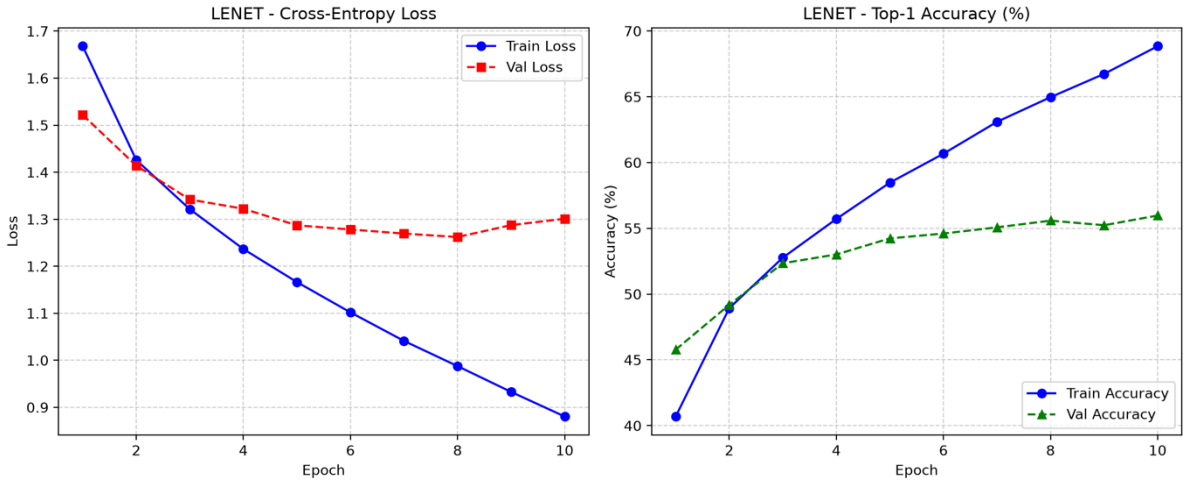


Figure 18: LeNet-5 Baseline

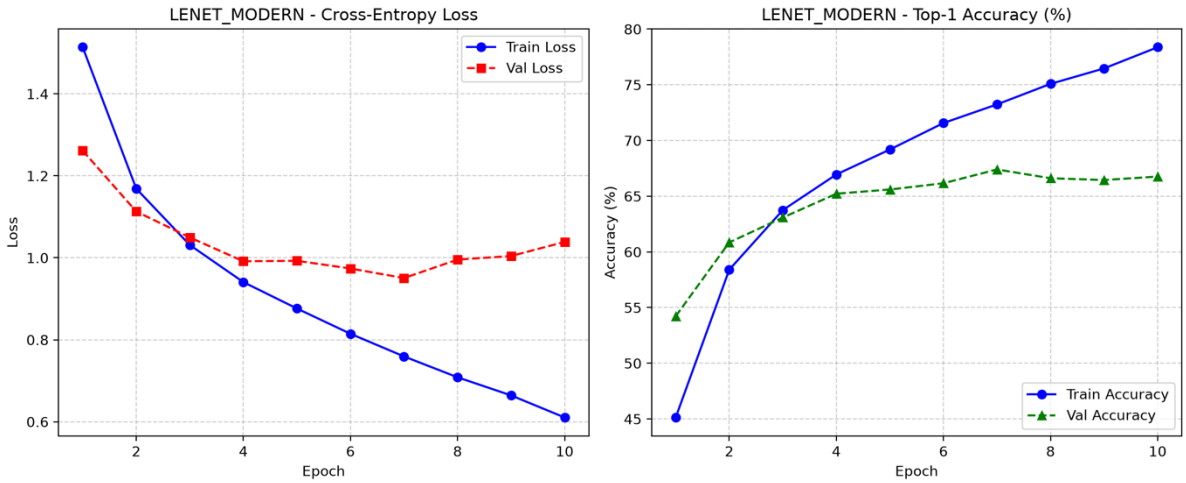


Figure 19: Modern LeNet

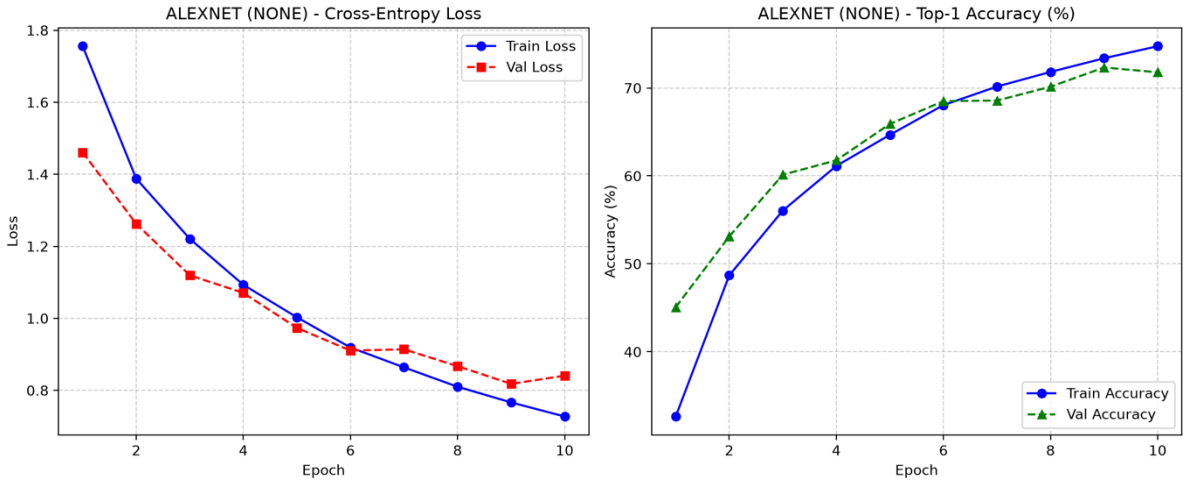


Figure 20: AlexNet None

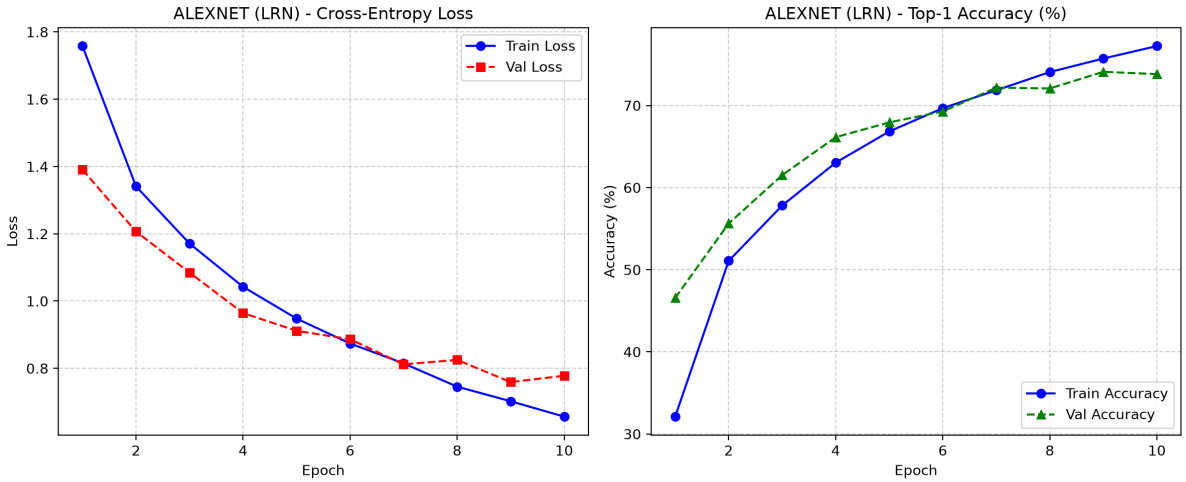


Figure 21: AlexNet LRN

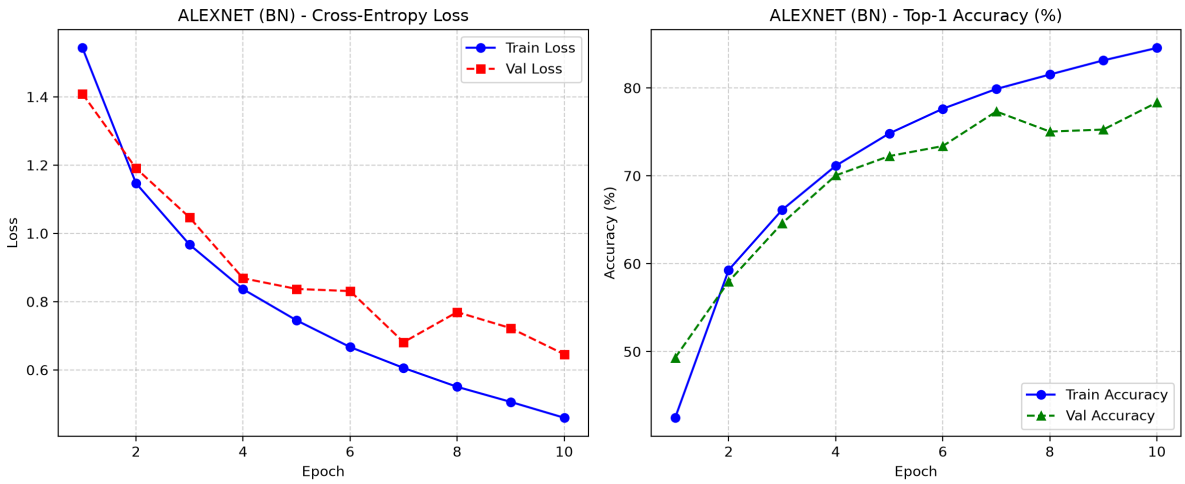


Figure 22: AlexNet BN

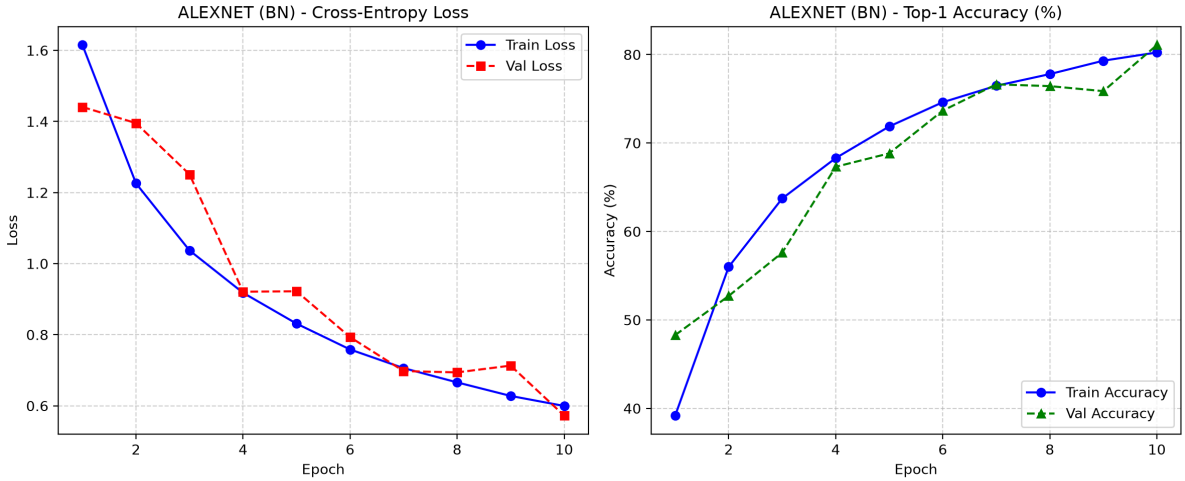


Figure 23: AlexNet BN Aug

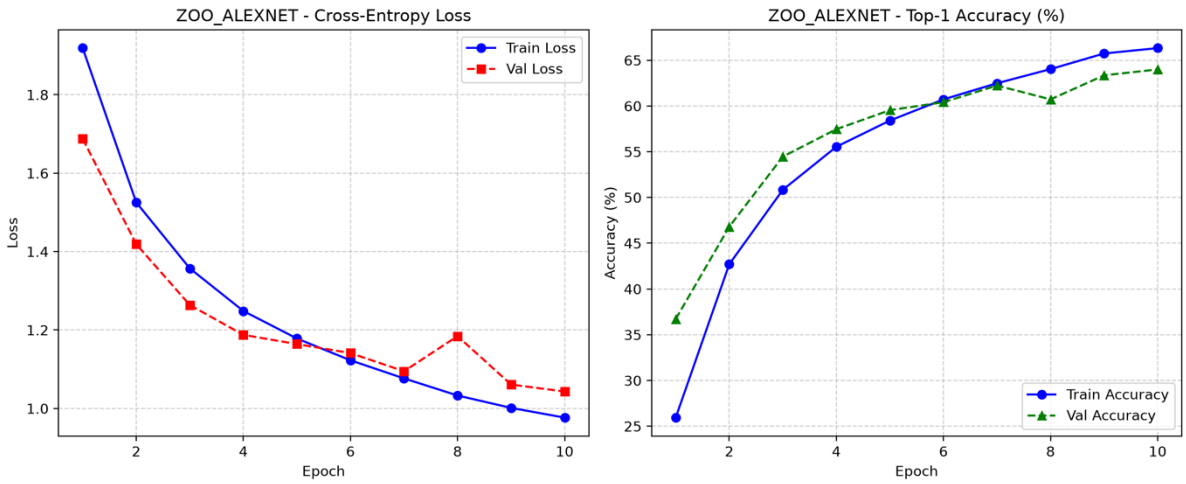


Figure 24: Zoo AlexNet

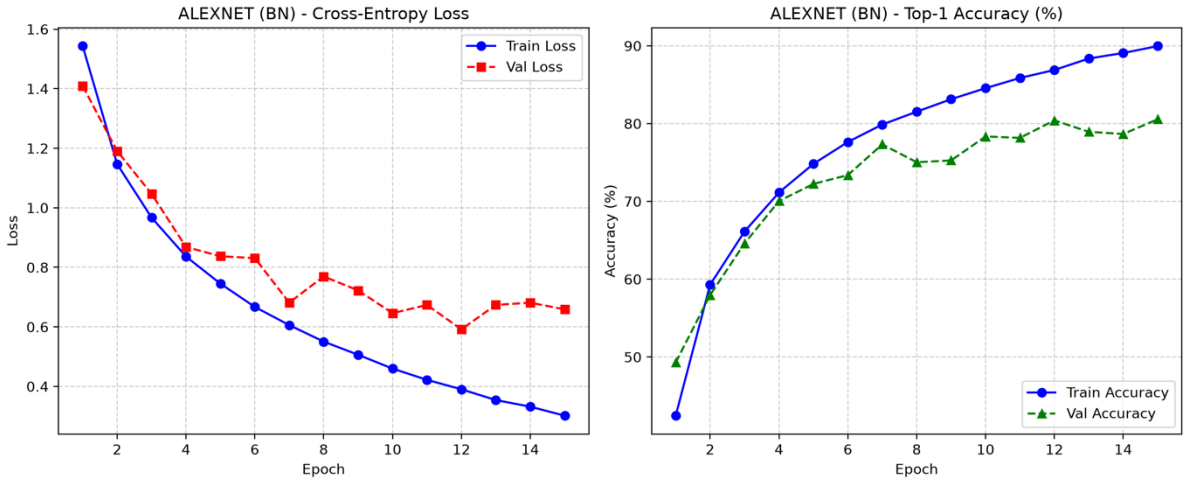


Figure 25: No Aug (AlexNet BN, 15 epochs)

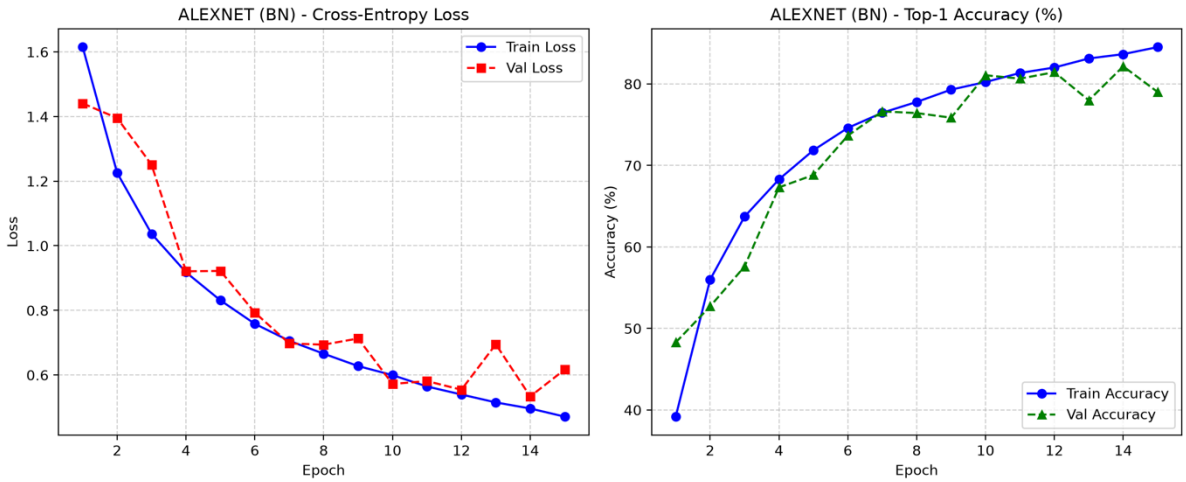


Figure 26: With Aug (AlexNet BN, 15 epochs)

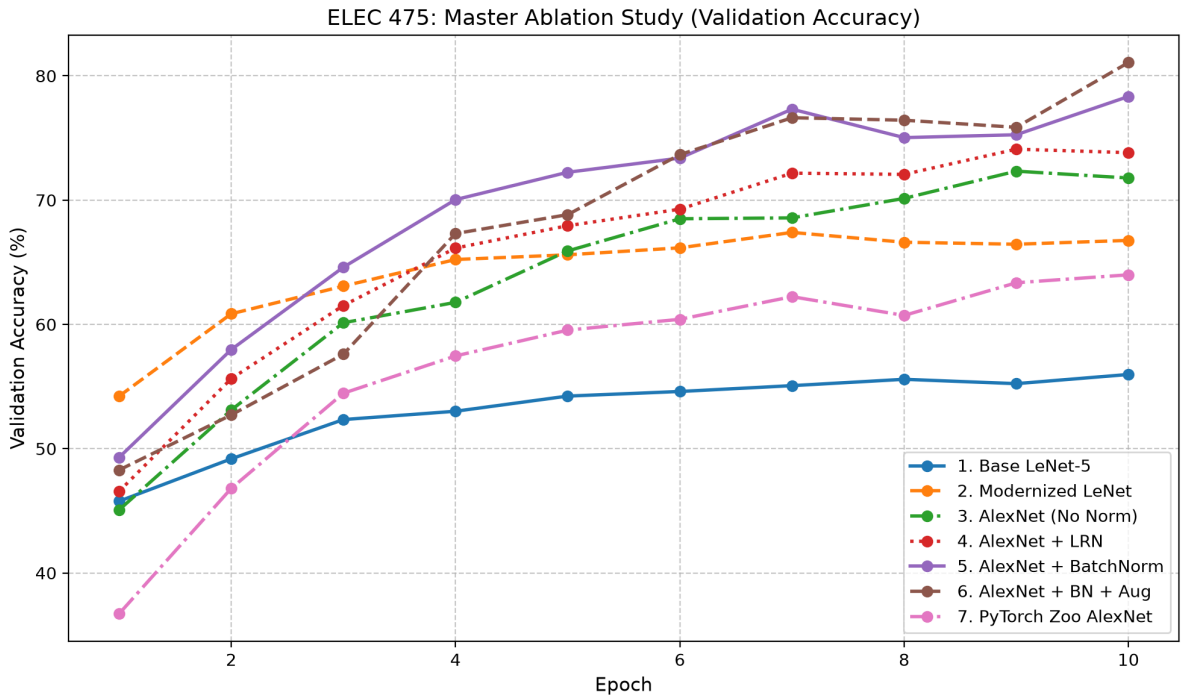


Figure 27: Ablation Summary

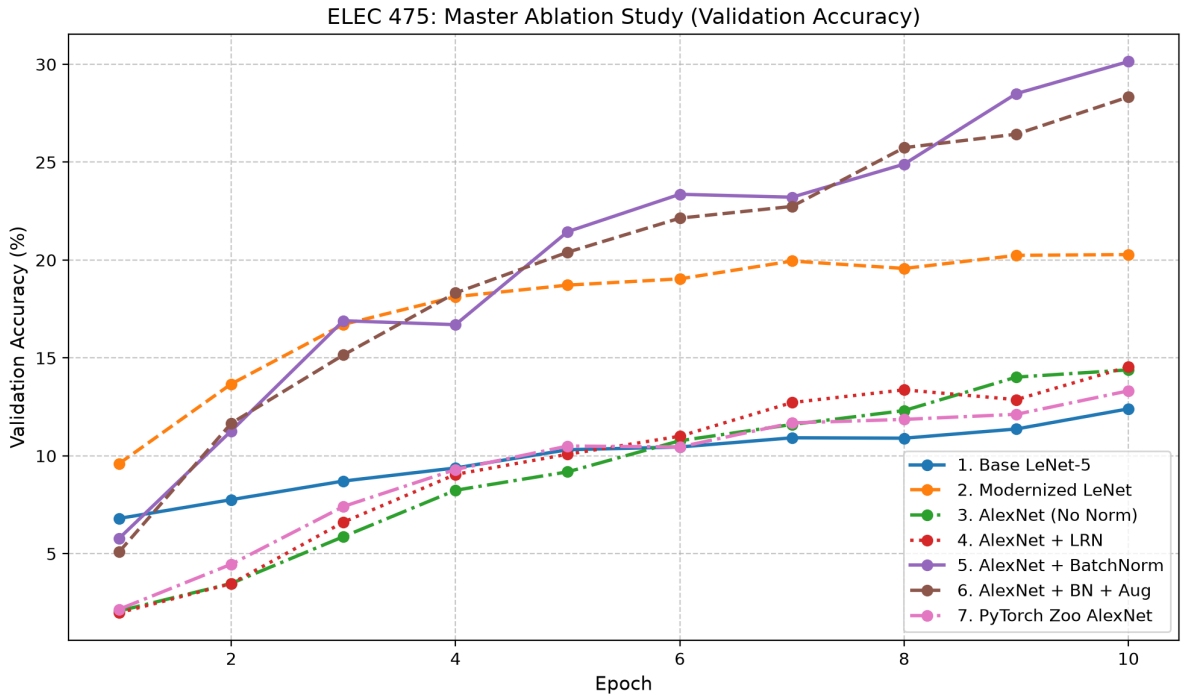


Figure 28: Ablation Summary Tiny ImageNet

3. Technical Analysis & Discussion

3.1 LeNet-5 vs. Modernized LeNet

Table 8: LeNet-5 vs. Modernized LeNet

Model	CIFAR-10	Tiny ImageNet	Final train acc (CIFAR-10)
LeNet-5	55.97	12.40	68.84
Modern LeNet	67.40 (+11.4)	20.28 (+7.9)	78.37

While Modern LeNet has 32% more parameters, it still has the same 0.26ms latency and only 2% more epoch time. The improvements are attributed to how ReLU does not saturate which allows gradients to stay strong, MaxPool keeps strongest activation instead of having it be diluted with the average, and overlapping windows with padding allows for more spatial detail to be retained (16x15x15 opposed to 16x13x13). These optimizations increase training and validation accuracy, but both models still overfit (final-epoch train-val gap of 12.9 points for LeNet-5 and 11.6 for Modern LeNet).

3.2 Normalization Study: None vs. LRN vs. BatchNorm

Table 9: Normalization study

Norm	CIFAR-10 val acc	Tiny ImageNet	Final train acc
None	72.08	14.39	74.75
LRN	74.07	14.55	77.23
BN	78.74	30.14	84.55

CIFAR-10 values are 3-seed means; Tiny ImageNet and final train accuracy are from the seed-42 run.

Wall-clock time: yes; FLOPs: essentially no. LRN adds 62% to epoch time (21.70 vs. 13.42s) and has the highest AlexNet-64 latency (0.91ms). The FLOPs barely change: LRN adds about 0 MFLOPs and BN adds about 1.3 MFLOPs (436.70 vs. 437.97), so the extra cost shows up as wall-clock time instead. BN adds only 11% to epoch time (14.94 vs. 13.42 s) while giving the largest accuracy gain, whereas LRN costs far more time for a much smaller gain. So yes, BatchNorm improved both validation accuracy and macro-F1 over None and LRN (78.74 vs. 74.07 and 72.08 accuracy; 78.66 vs. 74.00 and 71.89 macro-F1, Table 7). BN is better than LRN because it normalizes each channel with batch statistics, keeping activations scaled throughout all five conv layers. This

reduces internal covariate shift, where the input distribution of each layer keeps changing as the earlier layers update, and gives a smoother optimization landscape. This allows for faster and more stable optimization which is reflected in the higher training accuracy. LRN only normalizes neighbouring channels with fixed hyperparameters, leaving scale drift in deeper layers. The gap between LRN and BN is bigger on Tiny ImageNet because normalization matters more when the task gets harder. On Tiny ImageNet, LRN gives almost no benefit (14.55 vs. 14.39), while BN doubles accuracy.

For convergence stability, the training loss of all three schemes falls smoothly every epoch for all three seeds. BN reaches the lowest training loss (0.46 vs. 0.66 for LRN and 0.72 for None at epoch 10), and BN and None vary the least between seeds (std of about 0.01 vs. 0.02 for LRN). However, BN has the noisiest validation loss, with jumps of up to 0.14 between epochs, while LRN and None move by less than 0.03. This is likely because BN uses running averages of the batch statistics at evaluation time, which lag behind the quickly changing weights. Overall, BN was the most stable scheme in training, and LRN had the smoothest validation loss.

3.3 Effect of Data Augmentation on the Generalization Gap

Table 10: AlexNet BN with and without augmentation (15 epochs, CIFAR-10)

	No aug	**With aug**
Final train acc	89.95	84.52
Final val acc	80.55	79.01
Train−val gap	9.4 pts	5.5 pts
Best val acc	80.55	82.16
Best val macro-F1	80.53	82.13

These are separate 15-epoch runs, so they differ from the 10-epoch BN and BN + Aug runs in Table 5. The train−val gap uses final-epoch values.

The augmentation used is a random crop with 4-pixel padding, a horizontal flip, and ColorJitter (0.2). These transforms are label-preserving, since a flipped, shifted, or recoloured image still shows the same object, so the network cannot memorize where things sit or what exact colours they have in the training images and has to learn features that are invariant to these changes. So yes, augmentation reduced the generalization gap Δ from 9.4 to 5.5 points. Without augmentation, val loss is lowest at epoch 12 (0.59) and then rises while training accuracy increases which is overfitting.

With augmentation, val loss continues to fall and over 30 epochs (extended run, Table 6) it reaches 0.45 and 85.46%. So yes, augmentation improved validation performance: best val accuracy rose from 80.55 to 82.16% and best macro-F1 from 80.53 to 82.13%. We judge by the best epoch because the final val accuracy (79.01%) is one noisy epoch, since it was 82.16% at epoch 14.

Tiny ImageNet with its BN and Aug scores 28.33 vs. 30.14 for BN. The Tiny models are underfitting since AlexNet BN's final train and val accuracy are similar (30.5 vs. 30.1). Augmentation reduces overfitting, but with no overfitting to remove, it impedes training and 10 epochs is not enough to recover.

This also costs 43% more epoch time (21.38 vs 14.94s) because the CPU applies the random transforms to each image during data loading.

3.4 CIFAR-10 vs. Tiny ImageNet

From CIFAR-10 to Tiny ImageNet, every model drops by 44-60 pts. The best at classification is AlexNet BN which scored 30.14% on Tiny ImageNet and 78.34% on CIFAR-10. Random-chance accuracy on CIFAR-10 is 10% and 0.5% on Tiny. So yes, AlexNet BN is well above random chance on Tiny ImageNet (30.14% vs. 0.5%, about 60 times chance). Tiny ImageNet scored lower because there are 20 times the classes and 10 times fewer images per class (500 vs. 5,000), and more fine-grained classes that are hard to tell apart at 64x64. This also almost doubled the epoch time (27.8 vs. 14.9 s per epoch for AlexNet BN) because there are twice the images. One exception to the ranking is Modern LeNet (20.28%) beating AlexNet None and LRN (about 14.5%), since without BN the deeper network cannot train well in 10 epochs. Tiny ImageNet's val loss also did not diverge or plateau earlier than CIFAR-10's. It falls almost every epoch from 4.48 to 3.02, with only two small bumps at epochs 4 and 7, while CIFAR-10's val loss gets noisy and bottoms out around epochs 10-12 (Figure 29). The Tiny models are still improving and underfitting after 10 epochs. The gap between accuracy and macro-F1 is also wider on Tiny ImageNet (1.3-2.7 points vs. under 0.4 on CIFAR-10), since with 200 classes and only 50 validation images each, some classes are rarely predicted correctly, which pulls the macro average down.

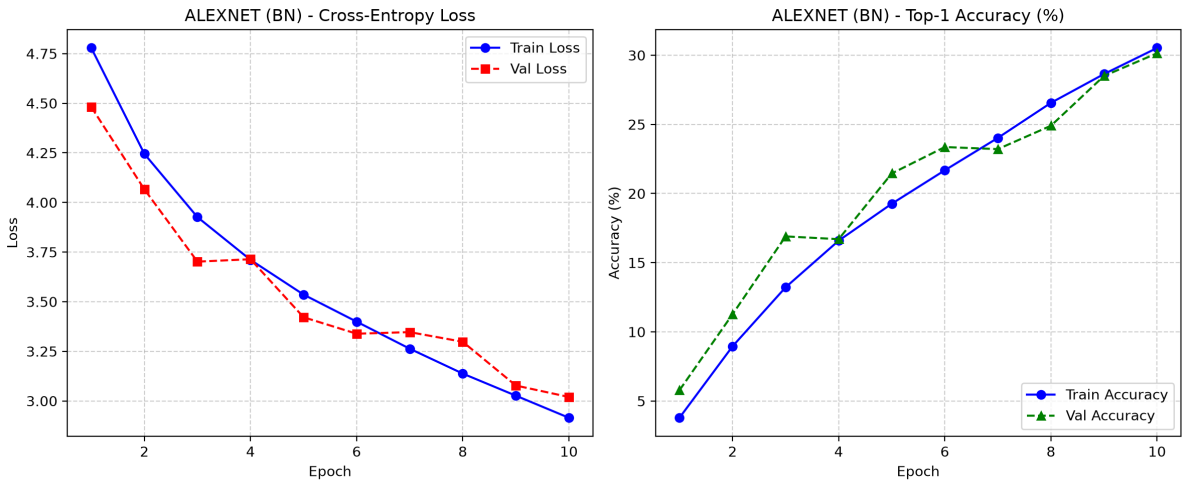


Figure 29: AlexNet BN training curves (Tiny ImageNet)

3.5 Accuracy vs. Cost Trade-offs (parameters, FLOPs, latency)

The table below shows that parameter count is not a predictor for accuracy. The zoo AlexNet (57 million params) has 8.6 times more parameters than AlexNet-64 BN (6.6 million params) and twice its latency (1.63 vs. 0.76 ms) but it scores lower on both CIFAR-10 (-14 points) and Tiny ImageNet (-17 points). It also uses fewer FLOPs (188 million vs. 438 million). This is because zoo AlexNet's stride-4 first layer collapses the 64x64 input to a 1x1 map before the classifier. As a result, about 95% of its parameters are in FC layers that receive the same repeated values and are hard to train from scratch in 10 epochs. The cheapest reasonable model is Modern LeNet because it has 0.45M params, costs 9 MFLOPs, and has a latency of 0.26 ms at 67% accuracy. AlexNet-64 BN gives the best accuracy among the AlexNet variants for almost no extra FLOPs, and its BN layers only cost 2,560 parameters and about 0.05 ms of latency. The least valuable variant is LRN where it adds 62% epoch time and the highest AlexNet-64 latency for only a gain of 2 points. A bonus is that augmentation adds training time but no inference cost, so it is free at deployment.

Table 11: Accuracy vs. cost

Model	Params	MFLOPs	Latency	CIFAR-10	Tiny
AlexNet-64 BN	6.6 M	438	0.76 ms	78.34	30.14
Zoo AlexNet	57.0 M	188	1.63 ms	63.99	13.32
LeNets	0.34–0.45 M	7–9	0.26 ms	56–67	12–20

4. Conclusion

In conclusion, adding normalization had the largest single effect: BatchNorm improved on AlexNet-64 by 6.7 points on CIFAR-10 and on Tiny ImageNet, accuracy doubled, while LRN gave little benefit at a high cost. Switching Tanh and average pooling to ReLU and overlapping max pooling increased LeNet's performance by 11 points at minimum cost. Data augmentation cuts the generalization gap by about 40% (9.4 to 5.5 points) and, combined with longer training, produces the best result of 85.46%. The zoo AlexNet showed that architecture must match its input: even though it has 9 times the parameters, it underperformed when given 64x64 images, which are smaller than the 224x224 input it expects. Training longer on Tiny ImageNet would reduce the underfitting, and the zoo AlexNet could benefit from pretrained weights.